Phase 2: Core Update Operations in Array ADT

1. Big Idea of Phase 2

Phase 2 teaches the first important operations of the Array ADT.

These operations are:

1. Display
2. Append
3. Insert
4. Delete

In Phase 1, we learned how an Array ADT stores data using three main parts:

```
struct Array {
    int A[20];
    int size;
    int length;
};
```

In Phase 2, we start using this structure to actually manage array elements.

The main idea of Phase 2 is:

```
Use length to know where valid data starts and ends.
Use size only to know the total capacity.
```

A normal array has memory slots, but the Array ADT adds rules so we can safely add, insert, remove, and display values.

---

2. Quick Review: size vs length

Before learning Phase 2 operations, this idea must be clear.

```
size = total capacity of the array
length = number of valid elements currently stored
```

Example:

```
struct Array arr = {{2, 3, 4, 5, 6}, 20, 5};
```

Memory view:

```
Index:  0  1  2  3  4  5  6  ... 19
Value:  2  3  4  5  6  _  _  ... _
```

Here:

```
size = 20
length = 5
```

That means:

```
Valid elements are from index 0 to index 4.
Index 5 is the first free position.
```

Important rule:

```
The last valid element is at index length - 1.
The first free slot is at index length.
```

For this example:

```
length = 5
last valid index = 4
first free index = 5
```

---

3. Why Phase 2 Uses the Fixed-Size Array Version

In Phase 1, we saw a dynamic version:

```
struct Array {
    int *A;
    int size;
```

```
    int length;
};
```

But in Phase 2, we usually use this fixed-size version:

```
struct Array {
    int A[20];
    int size;
    int length;
};
```

This version is easier for beginners because:

- The array is directly inside the structure.
- It is easier to see values in memory.
- It is easier to trace operations step by step.
- It avoids extra confusion about heap memory and pointers to heap arrays.

The operations work with the same logic whether the array is fixed-size or dynamically allocated. The main difference is only how the storage is created.

---

4. Display Operation

4.1 What Display Means

Display means printing all valid elements of the array.

It does not change the array.

It only reads and prints values.

Example:

```
Array: [2, 3, 4, 5, 6]
```

Display output:

```
2 3 4 5 6
```

4.2 Display Must Use length, Not size

Suppose:

```
size = 20
length = 5
```

Only 5 elements are valid.

So display must print only indexes:

```
0, 1, 2, 3, 4
```

Wrong idea:

```
for (int i = 0; i < arr.size; i++) {
    printf("%d ", arr.A[i]);
}
```

This is wrong because it prints all capacity slots, including unused slots.

Correct idea:

```
for (int i = 0; i < arr.length; i++) {
    printf("%d ", arr.A[i]);
}
```

This prints only valid logical elements.

4.3 Display Code

```
void display(struct Array arr) {
    for (int i = 0; i < arr.length; i++) {
        printf("%d ", arr.A[i]);
    }
    printf("\n");
}
```

4.4 Why Display Receives the Structure by Value

Display does not modify:

- array values
- size
- length

So it can receive the structure by value:

```
void display(struct Array arr)
```

This means display gets a copy of the structure information and only reads it.

Later operations like append, insert, and delete must receive a pointer because they modify the real array.

4.5 Display Dry Run

Given:

```
A = [2, 3, 4, 5, 6]
length = 5
```

Code:

```
for (int i = 0; i < arr.length; i++) {
    printf("%d ", arr.A[i]);
}
```

Trace:

Step	i	Condition `i < length`	A[i]	Action
1	0	true	2	print 2
2	1	true	3	print 3
3	2	true	4	print 4
4	3	true	5	print 5
5	4	true	6	print 6
6	5	false	-	stop

Final output:

```
2 3 4 5 6
```

4.6 Display Complexity

Display visits each valid element once.

If there are `N` valid elements:

```
Time complexity = O(N)
Space complexity = O(1)
```

Display uses constant extra space because it only needs a loop variable like `i`.

---

5. Append Operation

5.1 What Append Means

Append means adding a new value at the end of the current logical array.

The key rule is:

```
Append always inserts at index length.
```

Why?

Because index `length` is the first free slot.

Example:

```
A = [2, 3, 4, 5, 6]
length = 5
```

Valid elements are:

```
A[0] to A[4]
```

So the first free slot is:

```
A[5]
```

If we append `10`, we place it at `A[5]`.

5.2 Append Memory View

Before append:

```
Index:  0  1  2  3  4  5
Value:  2  3  4  5  6  _
length = 5
```

Append value:

```
10
```

Place `10` at index `length`, which is index `5`.

After append:

```
Index:  0  1  2  3  4  5
Value:  2  3  4  5  6  10
length = 6
```

5.3 Append Code

```c
void append(struct Array *arr, int x) {
    if (arr->length < arr->size) {
        arr->A[arr->length] = x;
        arr->length++;
    }
}
```

5.4 Why Append Needs a Pointer

Append modifies the real array.

It changes two things:

1. It places a new value into the array.
2. It increases `length`.

So the function must receive the address of the structure:

```
void append(struct Array *arr, int x)
```

Because `arr` is a pointer, we use the arrow operator:

```
arr->length
arr->size
arr->A[i]
```

Not dot:

```
arr.length    // wrong when arr is a pointer
```

5.5 Append Capacity Check

Before appending, we must check whether there is free space.

Condition:

```
if (arr->length < arr->size)
```

Meaning:

```
If current valid elements are less than total capacity, there is still space.
```

Example:

```
size = 20
length = 5
```

Since `5 < 20`, append is allowed.

But if:

```
size = 20
length = 20
```

Then the array is full. Append should not happen.

5.6 Append Dry Run

Starting array:

```
A = [2, 3, 4, 5, 6]
size = 20
length = 5
```

Call:

```
append(&arr, 10);
```

Trace:

Step	Action	Array	length
1	Check `length < size`	`[2, 3, 4, 5, 6]`	5
2	`5 < 20` is true	`[2, 3, 4, 5, 6]`	5
3	Store `10` at `A[length]`, which is `A[5]`	`[2, 3, 4, 5, 6, 10]`	5
4	Increment `length`	`[2, 3, 4, 5, 6, 10]`	6

Final array:

```
[2, 3, 4, 5, 6, 10]
```

5.7 Append Complexity

Append does not use a loop.

It only:

1. Checks capacity.
2. Stores the value.
3. Increases length.

So:

```
Time complexity = O(1)
Space complexity = O(1)
```

Append is constant time.

---

6. Insert Operation

6.1 What Insert Means

Insert means placing a value at a specific index.

Example:

```
A = [2, 3, 4, 5, 6]
```

Call:

```
insert(&arr, 2, 10);
```

Meaning:

```
Insert 10 at index 2.
```

But index `2` already contains `4`.

So we cannot simply place `10` there, because that would destroy the value `4`.

Wrong result:

```
[2, 3, 10, 5, 6]
```

The value `4` is lost.

Correct result:

```
[2, 3, 10, 4, 5, 6]
```

To get the correct result, we must shift elements to the right first.

6.2 Insert Main Idea

Insert follows this process:

1. Check whether the index is valid.
2. Shift elements right to create space.
3. Put the new value at the target index.
4. Increase `length`.

Simple rule:

```
Insert needs right shifting.
```

6.3 Valid Index for Insert

The valid insert condition is:

```
index >= 0 && index <= arr->length
```

This means insert is allowed from:

```
0 to length
```

Why is `index == length` allowed?

Because inserting at `length` means inserting at the end.

That is the same as append.

Example:

```
A = [2, 3, 4, 5, 6]
length = 5
```

This is valid:

```
insert(&arr, 5, 10);
```

Result:

```
[2, 3, 4, 5, 6, 10]
```

But this is invalid:

```
insert(&arr, 9, 10);
```

Because index `9` creates a gap between the current last element and the new element.

Arrays should remain compact.

6.4 Insert Code

```c
void insert(struct Array *arr, int index, int x) {
    if (index >= 0 && index <= arr->length) {
        for (int i = arr->length; i > index; i--) {
            arr->A[i] = arr->A[i - 1];
        }

        arr->A[index] = x;
        arr->length++;
    }
}
```

6.5 Important Note About Capacity in Insert

A safer insert function should also check whether the array has free space.

Better condition:

```
if (index >= 0 && index <= arr->length && arr->length < arr->size)
```

This prevents insertion when the array is full.

Improved version:

```
void insert(struct Array *arr, int index, int x) {
    if (index >= 0 && index <= arr->length && arr->length < arr->size) {
        for (int i = arr->length; i > index; i--) {
            arr->A[i] = arr->A[i - 1];
        }

        arr->A[index] = x;
        arr->length++;
    }
}
```

This version is safer because it checks both:

```
Is the index valid?
Is there enough space?
```

6.6 Why Insert Shifts from Right to Left

This is one of the most important Phase 2 ideas.

When inserting, shifting must start from the end.

Correct direction:

```
right to left
```

Code:

```
for (int i = arr->length; i > index; i--) {
    arr->A[i] = arr->A[i - 1];
}
```

Why?

Because if we shift from left to right, we overwrite values before moving them.

6.7 Wrong Left-to-Right Insert Example

Starting array:

```
[2, 3, 4, 5, 6]
```

Insert `10` at index `2`.

Wrong shift from front:

```
A[3] = A[2];
```

Now array becomes:

```
[2, 3, 4, 4, 6]
```

The original `5` at index `3` is lost.

Then if we continue, more values may be overwritten.

That is why insert must shift from the end backward.

6.8 Correct Insert Dry Run: Insert in the Middle

Starting array:

```
A = [2, 3, 4, 5, 6]
size = 20
length = 5
```

Call:

```
insert(&arr, 2, 10);
```

Target:

```
Insert 10 at index 2.
```

Before:

```
Index:  0  1  2  3  4  5
Value:  2  3  4  5  6  _
```

Trace:

Step	i	Action	Array state
Start	-	Original array	`[2, 3, 4, 5, 6, _]`
1	5	`A[5] = A[4]`	`[2, 3, 4, 5, 6, 6]`
2	4	`A[4] = A[3]`	`[2, 3, 4, 5, 5, 6]`
3	3	`A[3] = A[2]`	`[2, 3, 4, 4, 5, 6]`
4	2	Stop loop	`[2, 3, 4, 4, 5, 6]`
5	-	`A[2] = 10`	`[2, 3, 10, 4, 5, 6]`
6	-	`length++`	length becomes 6

Final result:

```
[2, 3, 10, 4, 5, 6]
```

6.9 Insert at the End

Starting array:

```
A = [2, 3, 4, 5, 6]
length = 5
```

Call:

```
insert(&arr, 5, 10);
```

Since `index == length`, there are no elements to shift.

The loop condition:

```
i > index
```

At the start:

```
i = 5
index = 5
```

So:

```
5 > 5 is false
```

No shifting happens.

Then:

```
arr->A[5] = 10;
arr->length++;
```

Result:

```
[2, 3, 4, 5, 6, 10]
```

This is basically append.

6.10 Insert at the Beginning

Starting array:

```
A = [2, 3, 4, 5, 6]
length = 5
```

Call:

```
insert(&arr, 0, 10);
```

Every element must shift right.

Trace:

```
A[5] = A[4]
A[4] = A[3]
A[3] = A[2]
A[2] = A[1]
A[1] = A[0]
A[0] = 10
```

Final result:

```
[10, 2, 3, 4, 5, 6]
```

This is the worst case for insert because all existing elements move.

6.11 Insert Invalid Cases

Starting array:

```
A = [2, 3, 4, 5, 6]
length = 5
```

Invalid call:

```
insert(&arr, -1, 10);
```

Reason:

```
Index cannot be negative.
```

Invalid call:

```
insert(&arr, 9, 10);
```

Reason:

```
Index is greater than length.
It would create empty holes.
```

For invalid insert, the function should do nothing.

This protects the array from partial or incorrect changes.

6.12 Insert Complexity

Best case:

```
Insert at the end.
```

No shifting is needed.

```
Time complexity = O(1)
```

Worst case:

```
Insert at the beginning.
```

All existing elements must shift right.

```
Time complexity = O(N)
```

Space complexity:

```
O(1)
```

Insert uses only a few extra variables.

---

7. Delete Operation

7.1 What Delete Means

Delete means removing the element at a given index.

Example:

```
A = [2, 3, 4, 5, 6]
```

Call:

```
Delete(&arr, 2);
```

This means:

```
Delete the value at index 2.
```

The value at index `2` is:

```
4
```

So after deletion, the logical array should become:

```
[2, 3, 5, 6]
```

7.2 Delete Must Not Leave Holes

Wrong idea:

```
[2, 3, _, 5, 6]
```

This leaves a blank space in the middle.

That is bad because the array is no longer compact.

Operations like display, search, and insert become more complicated if holes are allowed.

Correct idea:

```
Shift elements left to close the gap.
```

Correct result:

```
[2, 3, 5, 6]
```

7.3 Delete Main Idea

Delete follows this process:

1. Check whether the index is valid.
2. Save the deleted value.
3. Shift elements left to close the gap.
4. Decrease `length`.
5. Return the deleted value.

Simple rule:

```
Delete needs left shifting.
```

7.4 Valid Index for Delete

The valid delete condition is:

```
index >= 0 && index < arr->length
```

Delete is allowed only for existing elements.

If:

```
length = 5
```

Then valid indexes are:

```
0, 1, 2, 3, 4
```

Index `5` is not valid because it is the first free slot.

So this is invalid:

```
Delete(&arr, 5);
```

Insert allows `index == length`, but delete does not.

Important comparison:

Operation	Valid index range	Why
Insert	`0 <= index <= length`	Can insert at the end
Delete	`0 <= index < length`	Can delete only existing values

7.5 Delete Code

```c
int Delete(struct Array *arr, int index) {
    int x = 0;

    if (index >= 0 && index < arr->length) {
        x = arr->A[index];

        for (int i = index; i < arr->length - 1; i++) {
            arr->A[i] = arr->A[i + 1];
        }

        arr->length--;
    }
```

```
    return x;
}
```

7.6 Why Delete Saves the Value First

Before shifting, we save the value being deleted:

```
x = arr->A[index];
```

Why?

Because shifting will overwrite that value.

Example:

```
A = [2, 3, 4, 5, 6]
```

Delete index `2`.

If we do not save `A[2]`, then after shifting:

```
A[2] becomes 5
```

The original deleted value `4` is gone.

So we save it first:

```
x = 4
```

Then we can return it later.

7.7 Why Delete Shifts from Left to Right

When deleting, we must close the gap by moving right-side values left.

Code:

```c
for (int i = index; i < arr->length - 1; i++) {
    arr->A[i] = arr->A[i + 1];
}
```

This starts at the deleted index and copies the next value into the current position.

Example:

```
A[2] = A[3]
A[3] = A[4]
```

This removes the gap.

7.8 Why the Delete Loop Stops at length - 1

The loop condition is:

```c
i < arr->length - 1
```

Why not `i < arr->length`?

Because inside the loop we access:

```c
arr->A[i + 1]
```

If `i` reached the last valid index, then `i + 1` would go outside the valid array.

Example:

```
length = 5
last valid index = 4
```

If `i = 4`, then:

```
i + 1 = 5
```

But index `5` is not a valid element.

So the loop must stop before the last valid index.

7.9 Delete Dry Run: Delete from the Middle

Starting array:

```
A = [2, 3, 4, 5, 6]
length = 5
```

Call:

```
Delete(&arr, 2);
```

Target:

```
Delete value at index 2.
```

Before:

```
Index:  0  1  2  3  4
Value:  2  3  4  5  6
```

Trace:

Step	i	Action	Array state	Note
1	-	Save `x = A[2]`	`[2, 3, 4, 5, 6]`	`x = 4`
2	2	`A[2] = A[3]`	`[2, 3, 5, 5, 6]`	5 moves left
3	3	`A[3] = A[4]`	`[2, 3, 5, 6, 6]`	6 moves left
4	4	Stop loop	`[2, 3, 5, 6, 6]`	shifting complete
5	-	`length--`	logical array: `[2, 3, 5, 6]`	extra 6 ignored

Returned value:

```
4
```

Final logical array:

```
[2, 3, 5, 6]
```

7.10 Why the Duplicate at the End Does Not Matter

After deletion, we may see a duplicate value at the end.

Example after shifting:

```
[2, 3, 5, 6, 6]
```

But then we do:

```
arr->length--;
```

So if length becomes `4`, only these indexes are valid:

```
0, 1, 2, 3
```

The final `6` at index `4` is ignored.

Logical array:

```
[2, 3, 5, 6]
```

Important rule:

```
Values outside length do not matter.
```

7.11 Delete at the End

Starting array:

```
A = [2, 3, 4, 5, 6]
length = 5
```

Call:

```
Delete(&arr, 4);
```

Index `4` is the last valid index.

The value `6` is deleted.

No shifting is required because there are no elements after it.

Only length decreases:

```
length = 4
```

Final logical array:

```
[2, 3, 4, 5]
```

This is the best case for delete.

7.12 Delete at the Beginning

Starting array:

```
A = [2, 3, 4, 5, 6]
length = 5
```

Call:

```
Delete(&arr, 0);
```

The first element is deleted.

Every remaining element must shift left.

Trace:

```
x = A[0]
A[0] = A[1]
A[1] = A[2]
A[2] = A[3]
A[3] = A[4]
length--
```

Array state:

```
[3, 4, 5, 6, 6]
```

After reducing length:

```
[3, 4, 5, 6]
```

Returned value:

```
2
```

This is the worst case for delete because all remaining elements move.

7.13 Invalid Delete Cases

Starting array:

```
A = [2, 3, 4, 5, 6]
length = 5
```

Invalid call:

```
Delete(&arr, -1);
```

Reason:

```
Index cannot be negative.
```

Invalid call:

```
Delete(&arr, 5);
```

Reason:

```
Valid indexes are only 0 to 4.
Index 5 is outside the logical array.
```

When the index is invalid, the code returns `0` because:

```
int x = 0;
```

If the `if` condition fails, `x` remains `0`.

7.14 Delete Complexity

Best case:

```
Delete the last element.
```

No shifting is needed.

```
Time complexity = O(1)
```

Worst case:

```
Delete the first element.
```

All remaining elements shift left.

```
Time complexity = O(N)
```

Space complexity:

```
O(1)
```

Delete uses only a few extra variables.

8. Passing by Value vs Passing by Address in Phase 2

This is an important C concept used in Phase 2.

8.1 Pass by Value

Display uses pass by value:

```
void display(struct Array arr)
```

Why?

Because display only reads the array.

It does not modify the original structure.

8.2 Pass by Address

Append, insert, and delete use pass by address:

```
void append(struct Array *arr, int x)
void insert(struct Array *arr, int index, int x)
int Delete(struct Array *arr, int index)
```

Why?

Because these operations modify the real array.

They may change:

- actual values inside `A`
- `length`

If we passed the structure by value, changes would happen only to a copy.

The original array in `main()` would not be updated.

8.3 Dot Operator vs Arrow Operator

When using a normal structure variable:

```
arr.length
arr.size
arr.A[i]
```

When using a pointer to a structure:

```
arr->length
arr->size
arr->A[i]
```

In Phase 2:

Function	Parameter	Access style
display	`struct Array arr`	`arr.length`
append	`struct Array *arr`	`arr->length`
insert	`struct Array *arr`	`arr->length`
Delete	`struct Array *arr`	`arr->length`

---

9. Complete Phase 2 Program

```c
#include <stdio.h>

struct Array {
    int A[20];
    int size;
    int length;
};

void display(struct Array arr) {
    for (int i = 0; i < arr.length; i++) {
        printf("%d ", arr.A[i]);
    }
    printf("\n");
}
```

```c
void append(struct Array *arr, int x) {
    if (arr->length < arr->size) {
        arr->A[arr->length] = x;
        arr->length++;
    }
}

void insert(struct Array *arr, int index, int x) {
    if (index >= 0 && index <= arr->length && arr->length < arr->size) {
        for (int i = arr->length; i > index; i--) {
            arr->A[i] = arr->A[i - 1];
        }

        arr->A[index] = x;
        arr->length++;
    }
}

int Delete(struct Array *arr, int index) {
    int x = 0;

    if (index >= 0 && index < arr->length) {
        x = arr->A[index];

        for (int i = index; i < arr->length - 1; i++) {
            arr->A[i] = arr->A[i + 1];
        }

        arr->length--;
    }

    return x;
}

int main() {
    struct Array arr = {{2, 3, 4, 5, 6}, 20, 5};

    printf("Original array:\n");
    display(arr);

    append(&arr, 10);
    printf("After append 10:\n");
    display(arr);

    insert(&arr, 2, 15);
    printf("After inserting 15 at index 2:\n");
    display(arr);
```

```
    int deletedValue = Delete(&arr, 3);
    printf("Deleted value: %d\n", deletedValue);

    printf("After deletion:\n");
    display(arr);

    return 0;
}
```

9.1 Expected Output

Starting array:

```
[2, 3, 4, 5, 6]
```

After append `10`:

```
[2, 3, 4, 5, 6, 10]
```

After insert `15` at index `2`:

```
[2, 3, 15, 4, 5, 6, 10]
```

Then delete index `3`.

Value at index `3` is:

```
4
```

After deletion:

```
[2, 3, 15, 5, 6, 10]
```

Expected output:

```
Original array:
2 3 4 5 6
```

```
After append 10:
2 3 4 5 6 10
After inserting 15 at index 2:
2 3 15 4 5 6 10
Deleted value: 4
After deletion:
2 3 15 5 6 10
```

---

10. Operation Summary Table

Operation	Meaning	Changes array?	Main action	Time complexity
Display	Print valid elements	No	Traverse from `0` to `length - 1`	O(N)
Append	Add at end	Yes	Put value at `A[length]`	O(1)
Insert	Add at index	Yes	Shift right, place value	Best O(1), worst O(N)
Delete	Remove at index	Yes	Save value, shift left, reduce length	Best O(1), worst O(N)

---

11. Direction of Movement

This is a very useful way to remember Phase 2.

Operation	Direction	Why
Display	Left to right	Print each valid element
Append	Direct placement	First free slot is `length`
Insert	Right to left	Avoid overwriting existing values
Delete	Left to right	Close the gap after removing a value

---

12. Insert vs Delete Index Rules

This is a common place where beginners get confused.

Insert Rule

```
index >= 0 && index <= length
```

Insert allows `index == length` because inserting at the end is valid.

Example:

```
length = 5
valid insert indexes = 0, 1, 2, 3, 4, 5
```

Delete Rule

```
index >= 0 && index < length
```

Delete does not allow `index == length` because index `length` is not an existing element.

Example:

```
length = 5
valid delete indexes = 0, 1, 2, 3, 4
```

Final comparison:

```
Insert can happen at the first free slot.
Delete can happen only at an existing element.
```

---

13. Complexity Summary

Display

```
Time: O(N)
Space: O(1)
```

Reason:

```
It visits every valid element once.
```

Append

```
Time: O(1)
Space: O(1)
```

Reason:

```
It directly places the value at index length.
```

Insert

```
Best time: O(1)
Worst time: O(N)
Space: O(1)
```

Best case:

```
Insert at the end. No shifting.
```

Worst case:

```
Insert at the beginning. All elements shift right.
```

Delete

```
Best time: O(1)
Worst time: O(N)
Space: O(1)
```

Best case:

```
Delete the last element. No shifting.
```

Worst case:

```
Delete the first element. All remaining elements shift left.
```

---

14. Common Beginner Mistakes in Phase 2

Mistake 1: Printing up to size instead of length

Wrong:

```
for (int i = 0; i < arr.size; i++)
```

Correct:

```
for (int i = 0; i < arr.length; i++)
```

Use `length` when working with valid elements.

Mistake 2: Appending at size instead of length

Wrong:

```
arr->A[arr->size] = x;
```

Correct:

```
arr->A[arr->length] = x;
```

`length` gives the first free position.

Mistake 3: Forgetting to update length after append

Wrong:

```
arr->A[arr->length] = x;
```

Correct:

```
arr->A[arr->length] = x;
arr->length++;
```

If length is not increased, display will not show the new element.

Mistake 4: Forgetting to update length after insert

After inserting, always do:

```
arr->length++;
```

Otherwise, the inserted value may exist in memory but not be counted as part of the logical array.

Mistake 5: Shifting insert from left to right

Wrong:

```
for (int i = index; i < arr->length; i++) {
    arr->A[i + 1] = arr->A[i];
}
```

Correct:

```
for (int i = arr->length; i > index; i--) {
    arr->A[i] = arr->A[i - 1];
}
```

Insert must shift from the end to avoid losing values.

Mistake 6: Allowing delete at index == length

Wrong:

```
index <= arr->length
```

Correct:

```
index < arr->length
```

Delete can remove only existing elements.

Mistake 7: Forgetting to decrease length after delete

After shifting left, always do:

```
arr->length--;
```

Otherwise, the duplicate value at the end will still be treated as valid.

Mistake 8: Using dot instead of arrow with pointers

Wrong:

```
arr.length
```

Correct:

```
arr->length
```

When the function parameter is:

```
struct Array *arr
```

use `->`.

Mistake 9: Not checking capacity before append or insert

Before adding a new element, check:

```
arr->length < arr->size
```

If the array is full, do not add more values.

Mistake 10: Thinking unused slots are real elements

Unused slots may contain old values, duplicate values, or garbage values.

They should not be treated as part of the array.

Only indexes from `0` to `length - 1` are valid.

---

15. Beginner Memory Tricks

15.1 For Append

```
Append = Add at length.
```

Because `length` is the first free slot.

15.2 For Insert

```
Insert = Make space first, then put value.
```

Move elements right from the end.

15.3 For Delete

```
Delete = Save value, close gap, reduce length.
```

Move elements left.

15.4 For Display

```
Display = Print only real elements.
```

Loop only up to `length`.

---

16. Phase 2 Checkpoint Questions

You understand Phase 2 when you can answer these without looking:

1. What does display do?
2. Why does display use `length` instead of `size`?
3. What index does append use?
4. Why does append check `length < size`?
5. Why does append need a pointer to the structure?
6. Why does append increase `length`?
7. What does insert do?
8. Why does insert shift elements right?
9. Why must insert shift from the end backward?
10. What is the valid index condition for insert?
11. Why is `index == length` valid for insert?
12. What is the best case for insert?
13. What is the worst case for insert?
14. What does delete do?
15. Why does delete save the deleted value before shifting?
16. Why does delete shift elements left?
17. What is the valid index condition for delete?
18. Why is `index == length` invalid for delete?
19. Why does delete reduce `length`?
20. Why does the duplicate value at the end not matter after deletion?
21. Which Phase 2 functions modify the array?
22. Which Phase 2 function only reads the array?
23. When do we use `.` and when do we use `->`?
24. What is the time complexity of append?
25. What is the worst-case time complexity of insert and delete?

---

17. Phase 2 Final Summary

Phase 2 teaches how to update an Array ADT safely.

The four main operations are:

```
Display
Append
```

```
Insert
Delete
```

The most important rules are:

```
Display prints from index 0 to length - 1.
Append places the new value at index length.
Insert shifts elements right from the end, then places the value.
Delete saves the value, shifts elements left, then reduces length.
```

The main idea is to keep the array compact.

A compact array has no holes between valid elements.

Example of compact array:

```
[2, 3, 4, 5, 6]
```

Example of bad array with a hole:

```
[2, 3, _, 5, 6]
```

Phase 2 protects the array from three common problems:

1. Printing unused slots.
2. Overwriting values during insertion.
3. Leaving holes during deletion.

One-line memory:

```
Phase 2 is about displaying valid data, adding at the end, inserting by shifting
right, and deleting by shifting left.
```

After Phase 2, you are ready for Phase 3: Linear Search.